

See the Assessment Guide for information on how to interpret this report.

ASSESSMENT SUMMARY

Compilation:	PASSED
API:	PASSED
SpotBugs:	PASSED
PMD:	FAILED (1 warning)
Checkstyle:	FAILED (0 errors, 6 warnings)
Correctness:	33/33 tests passed
Memory:	No tests available for autograding.
Timing:	No tests available for autograding.
Aggregate score: 100.00%	
[Compilation: 5%, API: 5%, Style: 0%, Correctness: 90%]	

ASSESSMENT DETAILS

```
The following files were submitted:
-----
 904 Nov  4 19:54 Checkerboard.java
1.2K Nov  4 19:54 ShannonEntropy.java
1.2K Nov  4 19:54 WorldMap.java

*****
*   COMPILING
*****

% javac ShannonEntropy.java
*-----

% javac Checkerboard.java
*-----

% javac WorldMap.java
*-----

=====

Checking the APIs of your programs.
*-----
ShannonEntropy:

Checkerboard:

WorldMap:

=====

*****
*   CHECKING STYLE AND COMMON BUG PATTERNS
*****

% spotbugs *.class
*-----

=====

% pmd .
*-----
Checkerboard.java:13: Avoid unused local variables, such as 'k'. [UnusedLocalVariable]
PMD ends with 1 warning.

=====

% checkstyle *.java
*-----
[WARN] Checkerboard.java:21:15: '/' or '/' is not followed by whitespace. [WhitespaceAfter]
[WARN] ShannonEntropy.java:30:11: '/' or '/' is not followed by whitespace. [WhitespaceAfter]
Checkstyle ends with 0 errors and 2 warnings.

% custom checkstyle checks for ShannonEntropy.java
*-----
[WARN] ShannonEntropy.java:14:16: Using a 'double[]' array suggests poor design in this program. [DescendantToken]
Checkstyle ends with 0 errors and 1 warning.

% custom checkstyle checks for Checkerboard.java
*-----

% custom checkstyle checks for WorldMap.java
*-----
[WARN] WorldMap.java:7: Use 'StdIn.isEmpty()' to determine when no more input is available on standard input. [Design]
[WARN] WorldMap.java:7: Use 'StdIn.readString()' to read the region name. [Design]
[WARN] WorldMap.java:7: You will get substantially improved performance if you call 'StdDraw.enableDoubleBuffering()' once at the beginning if this program and 'StdDraw.show()' once at the end of this program. [Design]
Checkstyle ends with 0 errors and 3 warnings.

=====

*****
*   TESTING CORRECTNESS
*****

Testing correctness of ShannonEntropy
*-----
Running 12 total tests.

Test 1: check output format
  % java-introcs ShannonEntropy 2 < fair-coin.txt
  1.0000

  % java-introcs ShannonEntropy 6 < loaded-die.txt
  1.8750

  % java-introcs ShannonEntropy 100 < letters100.txt
  4.3798

==> passed

Test 2: check that program reads all data from standard input
  * java-introcs ShannonEntropy < fair-coin.txt
  * java-introcs ShannonEntropy < loaded-die.txt
  * java-introcs ShannonEntropy < letters100.txt
==> passed

Test 3: check correctness for fixed inputs
  * java-introcs ShannonEntropy 2 < fair-coin.txt
  * java-introcs ShannonEntropy 2 < biased-coin.txt
  * java-introcs ShannonEntropy 6 < loaded-die.txt
  * java-introcs ShannonEntropy 100 < letters100.txt
  * java-introcs ShannonEntropy 3 < roshambo.txt
==> passed

Test 4: check correctness when entropy is 0
  * m = 1, all 1s
  * m = 2, all 1s
```

```
* m = 2, all 2s
* m = 6, all 1s
* m = 6, all 3s
* m = 6, all 6s
* m = 100, all 100s
==> passed

Test 5: check correctness for random inputs with p_i > 0 for all i
* 10 random integers between 1 and 2
* 100 random integers between 1 and 2
* 20 random integers between 1 and 6
* 1000 random integers between 1 and 6
* 1000 random integers between 1 and 26
==> passed

Test 6: check correctness for random inputs with p_m = 0
* 10 random integers between 1 and 6
* 50 random integers between 1 and 10
==> passed

Test 7: check correctness for random inputs with p_i = 0 for some i
* 10 random integers between 1 and 6
* 100 random integers between 1 and 6
* 50 random integers between 1 and 26
* 1000 random integers between 1 and 26
==> passed

Test 8: check correctness for random inputs with m = n
* 2 random integers between 1 and 2
* 3 random integers between 1 and 3
* 4 random integers between 1 and 4
* 6 random integers between 1 and 6
==> passed

Test 9: check correctness for random inputs with m > n
* 3 random integers between 1 and 6
* 10 random integers between 1 and 26
==> passed

Test 10: check correctness for inputs with large n
* java-introcs ShannonEntropy 26 < letters.txt
==> passed

Test 11: check correctness for inputs with large m
* java-introcs ShannonEntropy 1000000 < ids20.txt
==> passed

Test 12: check correctness for inputs with large m and n
* java-introcs ShannonEntropy 1000000 < ids100000.txt
* java-introcs ShannonEntropy 1000000 < ids1000000.txt
==> passed

Total: 12/12 tests passed!
```

```
=====
Testing correctness of Checkerboard
*-----
Running 12 total tests.
```

```
Test 1: check output format
% java Checkerboard 1
[no output]

% java Checkerboard 2
[no output]

% java Checkerboard 3
[no output]

% java Checkerboard 4
[no output]

% java Checkerboard 5
[no output]

% java Checkerboard 6
[no output]

==> passed
```

```
Test 2: check formatting of standard drawing
* java Checkerboard 1
[no output]

* java Checkerboard 2
[no output]

* java Checkerboard 3
[no output]

* java Checkerboard 4
[no output]

* java Checkerboard 5
[no output]

* java Checkerboard 6
[no output]

==> passed
```

```
Test 3: check number of filled polygons
* n = 1
* n = 2
* n = 3
* n = 4
* n = 5
* n = 6
==> passed
```

```
Test 4: check correctness, ignoring color
* n = 1
* n = 2
* n = 3
* n = 4
* n = 5
* n = 6
==> passed
```

```
Test 5: check that lower-left square is blue for odd n
* n = 1
* n = 3
* n = 5
* n = 7
* n = 9
==> passed
```

```
Test 6: check that lower-left square is blue for even n
* n = 2
* n = 4
* n = 6
* n = 8
* n = 10
==> passed
```

```
Test 7: check that lower-left square is blue for large n
* 10 <= n < 20
* 20 <= n < 30
* 30 <= n < 50
==> passed
```

```
Test 8: check that it draws a checkerboard pattern for even n
* n = 2
* n = 4
* n = 6
* n = 8
* n = 10
==> passed
```

```
Test 9: check that it draws a checkerboard pattern for odd n
* n = 1
* n = 3
* n = 5
* n = 7
* n = 9
==> passed

Test 10: check that it draws a checkerboard pattern for larger n
* 10 <= n < 20
* 20 <= n < 30
* 30 <= n < 50
==> passed

Test 11: check number of calls to StdDraw.setXscale() and StdDraw.setYscale()
* n = 2
* n = 4
* n = 6
* n = 8
* n = 9
==> passed

Test 12: check arguments to StdDraw.setXscale() and StdDraw.setYscale()
* n = 2
* n = 4
* n = 6
* n = 8
* n = 9
==> passed
```

Checkerboard Total: 12/12 tests passed!

```
=====
Testing correctness of WorldMap
*-----
Running 9 total tests.
```

```
Test 1: check formatting of standard output
% java WorldMap < square.txt

% java WorldMap < shapes.txt

% java WorldMap < usa.txt

% java WorldMap < russia.txt

% java WorldMap < india.txt

% java WorldMap < world-low-resolution.txt
```

==> passed

```
Test 2: check that program reads all data from standard input
* java-introcs WorldMap < square.txt
* java-introcs WorldMap < shapes.txt
* java-introcs WorldMap < usa.txt
* java-introcs WorldMap < russia.txt
* java-introcs WorldMap < india.txt
==> passed
```

```
Test 3: check formatting of standard drawing
* java WorldMap < square.txt

* java WorldMap < shapes.txt

* java WorldMap < usa.txt

* java WorldMap < russia.txt

* java WorldMap < india.txt

* java WorldMap < world-low-resolution.txt
```

==> passed

```
Test 4: check number of polygons
* file = square.txt
* file = shapes.txt
* file = usa.txt
* file = russia.txt
* file = india.txt
* file = world-low-resolution.txt
==> passed
```

```
Test 5: check correctness for input files
* Square                square.txt
* Triange, Square, and Pentagon shapes.txt
* USA                   usa.txt
* Russia                russia.txt
* India                 india.txt
* World (low resolution) world-low-resolution.txt
==> passed
```

```
Test 6: check correctness for input files with one polygon
* Hilbert polygon of order 1  hilbert1.txt
* Hilbert polygon of order 2  hilbert2.txt
* Hilbert polygon of order 3  hilbert3.txt
* Hilbert polygon of order 4  hilbert4.txt
* Hilbert polygon of order 5  hilbert5.txt
* Hilbert polygon of order 6  hilbert6.txt
==> passed
```

```
Test 7: check correctness for random countries
* South Africa          world/south-africa.txt
* Haiti                 world/haiti.txt
* Angola                world/angola.txt
* Serbia Without Kosovo world/serbia-without-kosovo.txt
* Mali                  world/mali.txt
* Czech Republic        world/czech-republic.txt
* Finland               world/finland.txt
* Kenya                world/kenya.txt
* Bulgaria               world/bulgaria.txt
* Portugal Regions      world/portugal-regions.txt
* Paraguay               world/paraguay.txt
* Chile                  world/chile.txt
* Macedonia              world/macedonia.txt
* Brazil                 world/brazil.txt
* Moldova                world/moldova.txt
==> passed
```

```
Test 8: check correctness for random states in USA
* Georgia                usa/usa-ga.txt
* Oklahoma                usa/usa-ok.txt
* New Hampshire           usa/usa-nh.txt
* Alaska                  usa/usa-ak.txt
* Tennessee               usa/usa-tn.txt
* Washington              usa/usa-wa.txt
* Colorado                usa/usa-co.txt
* North Carolina          usa/usa-nc.txt
* Kansas                  usa/usa-ks.txt
* New Jersey               usa/usa-nj.txt
* Massachusetts           usa/usa-ma.txt
* Wyoming                 usa/usa-wy.txt
* California               usa/usa-ca.txt
* Maryland                 usa/usa-md.txt
* Illinois                 usa/usa-il.txt
==> passed
```

```
Test 9: check correctness of large input files
* USA (all counties)      usa-all-counties.txt
* World                   world.txt
==> passed
```

WorldMap Total: 9/9 tests passed!

=====